5CS037 - Concepts and Technologies of AI.
Worksheet - 1: A coding exercises on Numpy.

1 TO - DO - Task

Please complete all the problem listed below.

1.1 Warming Up Exercise: Basic Vector and Matrix Operation with Numpy.

Problem - 1: Array Creation:

Complete the following Tasks:

1. Initialize an empty array with size 2*X*2.

```
import numpy as np
import time
np.set_printoptions(suppress=True, precision=6)
```

```
arr_empty_2x2 = np.empty((2,2))
print("empty 2x2:\n", arr_empty_2x2)
```

```
empty 2x2:
 [[0. 0.]
 [0. 0.]]
```

2. Initialize an all one array with size 4*X*2.

```
arr_ones_4x2 = np.ones((4,2))
print("\nones 4x2:\n", arr_ones_4x2)
```

```

ones 4x2:
 [[1. 1.]
 [1. 1.]
 [1. 1.]
 [1. 1.]]
```

3. Return a new array of given shape and type, filled with fill value.{Hint: np.full}

```
arr_full_3x3_fill7 = np.full((3,3), 7, dtype=int)
print("\nfull 3x3 filled with 7:\n", arr_full_3x3_fill7)
```

```
full 3x3 filled with 7:
 [[7 7 7]
 [7 7 7]
 [7 7 7]]
```

4. Return a new array of zeros with same shape and type as a given array.{Hint: np.zeros like} -

```
sample = np.array([[1,2,3],[4,5,6]])
arr_zeros_like_sample = np.zeros_like(sample)
print("\nzeros_like sample:\n", arr_zeros_like_sample)
```

```
zeros_like sample:
 [[0 0 0]
 [0 0 0]]
```

5. Return a new array of ones with same shape and type as a given array.{Hint: np.ones like} -

```
arr_ones_like_sample = np.ones_like(sample)
print("\nones_like sample:\n", arr_ones_like_sample)
```

```
ones_like sample:
 [[1 1 1]
 [1 1 1]]
```

6. For an existing list new_list = [1,2,3,4] convert to an numpy array.{Hint: np.array()}

```
new_list = [1,2,3,4]
arr_from_list = np.array(new_list)
print("\narray from list:", arr_from_list)
```

```
array from list: [1 2 3 4]
```

Problem - 2: Array Manipulation: Numerical Ranges and Array indexing:

Complete the following tasks:

1. Create an array with values ranging from 10 to 49. {Hint:np.arrange()}.

```
arr_10_to_49 = np.arange(10,50)
print("10..49:", arr_10_to_49)
```

```
10..49: [10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33
 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49]
```

2. Create a 3*X*3 matrix with values ranging from 0 to 8.
   {Hint:look for np.reshape()}

```
mat_3x3_0_8 = np.arange(9).reshape((3,3))
print("\n3x3 matrix 0..8:\n", mat_3x3_0_8)
```

```
3x3 matrix 0..8:
 [[0 1 2]
 [3 4 5]
 [6 7 8]]
```

3. Create a 3*X*3 identity matrix.{Hint:np.eye()}

```
id_3x3 = np.eye(3, dtype=int)
print("\n3x3 identity:\n", id_3x3)
```

```
3x3 identity:
 [[1 0 0]
 [0 1 0]
 [0 0 1]]
```

4. Create a random array of size 30 and find the mean of the array.
   {Hint:check for np.random.random() and array.mean() function}

```
rand30 = np.random.random(30)
mean_rand30 = rand30.mean()
print("\nrandom(30) mean:", mean_rand30)
```

```
random(30) mean: 0.495238146786660006
```

5. Create a 10*X*10 array with random values and find the minimum and maximum values.

```
rand10x10 = np.random.random((10,10))
min_10x10 = rand10x10.min()
max_10x10 = rand10x10.max()
print("\n10x10 min:", min_10x10, "max:", max_10x10)
```

```
10x10 min: 0.003724436820827015 max: 0.9991674810211209
```

6. Create a zero array of size 10 and replace 5^{th} element with 1.

[13] ✓ 0s

```
zero10 = np.zeros(10, dtype=int)
zero10[4] = 1  # 5th element is index 4
print("\nzero10 with 5th element 1:", zero10)
```

```
zero10 with 5th element 1: [0 0 0 0 1 0 0 0 0 0]
```

7. Reverse an array arr = [1,2,0,0,4,0].

[14] ✓ 0s

```
arr = np.array([1,2,0,0,4,0])
arr_reversed = arr[::-1]
print("\noriginal:", arr, "reversed:", arr_reversed)
```

```
original: [1 2 0 0 4 0] reversed: [0 4 0 0 2 1]
```

8. Create a 2d array with 1 on border and 0 inside.

```python
def border_matrix(n):
    m = np.zeros((n,n), dtype=int)
    m[0,:] = 1
    m[-1,:] = 1
    m[:,0] = 1
    m[:,-1] = 1
    return m

border_5 = border_matrix(5)
print("\n5x5 border matrix:\n", border_5)
```

```
...

5x5 border matrix:
 [[1 1 1 1 1]
 [1 0 0 0 1]
 [1 0 0 0 1]
 [1 0 0 0 1]
 [1 1 1 1 1]]
```

9. Create a 8*X*8 matrix and fill it with a checkerboard pattern.

```python
checker = np.indices((8,8)).sum(axis=0) % 2
print("\n8x8 checkerboard:\n", checker)
```

```
...

8x8 checkerboard:
 [[0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]]
```

Problem - 3: Array Operations:

For the following arrays:

x = np.array([[1,2],[3,5]]) and y = np.array([[5,6],[7,8]]); v = np.array([9,10]) and w = np.array([11,12]);
Complete all the task using numpy:

1. Add the two array.

```
x = np.array([[1,2],[3,5]])
y = np.array([[5,6],[7,8]])
v = np.array([9,10])
w = np.array([11,12])
add_xy = x + y
print("x + y:\n", add_xy)
```

```
x + y:
 [[ 6  8]
 [10 13]]
```

```
sub_xy = x - y
print("\nx - y:\n", sub_xy)
```

2. Subtract the two array.

```
sub_xy = x - y
print("\nx - y:\n", sub_xy)
```

```
x - y:
 [[-4 -4]
 [-4 -3]]
```

3. Multiply the array with any integers of your choice.

```
mul_x3 = x * 3
print("\nx * 3:\n", mul_x3)
```

```
x * 3:
 [[ 3  6]
 [ 9 15]]
```

4. Find the square of each element of the array.

```
square_x = x**2
print("\nx squared:\n", square_x)
```

```
x squared:
 [[ 1  4]
 [ 9 25]]
```

5. Find the dot product between: v(and)w ; x(and)v ; x(and)y.

[21] ✓ 0s

```
dot_v_w = np.dot(v, w)
dot_x_v = x.dot(v)
dot_x_y = x.dot(y)
print("\nDot v.w:", dot_v_w)
print("Dot x.v:", dot_x_v)
print("Dot x.y:\n", dot_x_y)
```

```
Dot v.w: 219
Dot x.v: [29 77]
Dot x.y:
 [[19 22]
 [50 58]]
```

6. Concatenate x(and)y along row and Concatenate v(and)w along column.
   {Hint:try np.concatenate() or np.vstack() functions.

```
[22]
✓ 0s
```

```
concat_xy_axis0 = np.concatenate((x, y), axis=0)
concat_vw_col = np.column_stack((v, w))
print("\nconcat x,y along rows:\n", concat_xy_axis0)
print("\nconcat v,w as columns:\n", concat_vw_col)
```

```
...

concat x,y along rows:
 [[1 2]
 [3 5]
 [5 6]
 [7 8]]

concat v,w as columns:
 [[ 9 11]
 [10 12]]
```

7. Concatenate x(and)v; if you get an error, observe and explain why did you get the error?

```
try:
    concat_x_v = np.concatenate((x, v), axis=0)
    print("\nconcat x and v result:\n", concat_x_v)
except Exception as e:
    print("\nConcatenating x and v raised an error as expected:")
    print("Error:", e)
    print("Explanation: x has shape", x.shape, "but v has shape", v.shape,
          "— cannot concatenate along axis 0 because dimensions don't match.")
```

```
...
Concatenating x and v raised an error as expected:
Error: all the input arrays must have same number of dimensions, but the array at index 0 has 2 dimension(s) and the array at index 1 h
Explanation: x has shape (2, 2) but v has shape (2,) — cannot concatenate along axis 0 because dimensions don't match.
```

Problem - 4: Matrix Operations:

- For the following arrays:

  A = np.array([[3,4],[7,8]]) and B = np.array([[5,3],[2,1]]);
  Prove following with Numpy:

  1. Prove $A.A^{-1} = I$.

```
A = np.array([[3,4],[7,8]])
B = np.array([[5,3],[2,1]])
I = np.eye(2, dtype=float)

A_inv = np.linalg.inv(A)
prod_A_Ainv = A.dot(A_inv)
print("A * A_inv:\n", prod_A_Ainv)
```

```
A * A_inv:
 [[1. 0.]
 [0. 1.]]
```

2. Prove $AB \neq BA$.

[25] ✓ 0s

```
AB = A.dot(B)
BA = B.dot(A)
print("\nA*B:\n", AB)
print("\nB*A:\n", BA)
print("\nAre AB and BA equal?", np.allclose(AB, BA))
```

```
A*B:
 [[23 13]
 [51 29]]

B*A:
 [[36 44]
 [13 16]]

Are AB and BA equal? False
```

3. Prove $(AB)^T = B^T A^T$.

[28] ✓ 0s

```
lhs = (A.dot(B)).T
rhs = B.T.dot(A.T)
print("\n(AB)^T:\n", lhs)
print("\nB^T A^T:\n", rhs)
print("\n(AB)^T == B^T A^T ?", np.allclose(lhs, rhs))
coeff = np.array([[2, -3, 1],
                  [1, -1, 2],
                  [3,  1, -1]], dtype=float)
rhs_vec = np.array([-1, -3, 9], dtype=float)

solution = np.linalg.solve(coeff, rhs_vec)
print("\nSolution (using np.linalg.solve):", solution)

coeff_inv = np.linalg.inv(coeff)
solution_via_inv = coeff_inv.dot(rhs_vec)
print("Solution (via inverse):", solution_via_inv)

print("Solutions equal?", np.allclose(solution, solution_via_inv))
```

```
(AB)^T:
 [[23 51]
 [13 29]]

B^T A^T:
 [[23 51]
 [13 29]]

(AB)^T == B^T A^T ? True

Solution (using np.linalg.solve): [ 2.  1. -2.]
Solution (via inverse): [ 2.  1. -2.]
Solutions equal? True
```

- Solve the following system of Linear equation using Inverse Methods.

$$2x - 3y + z = -1\,x$$
$$-y + 2z = -3\,3x$$
$$+y - z = 9$$

{Hint: First use Numpy array to represent the equation in Matrix form. Then Solve for: $AX = B$}

- Now: solve the above equation using np.linalg.inv function.{Explore more about "linalg" function of Numpy}

```
coeff = np.array([[2, -3, 1],
                  [1, -1, 2],
                  [3,  1, -1]], dtype=float)
rhs_vec = np.array([-1, -3, 9], dtype=float)

solution = np.linalg.solve(coeff, rhs_vec)
print("\nSolution (using np.linalg.solve):", solution)

coeff_inv = np.linalg.inv(coeff)
solution_via_inv = coeff_inv.dot(rhs_vec)
print("Solution (via inverse):", solution_via_inv)

print("Solutions equal?", np.allclose(solution, solution_via_inv))
```

```
...
Solution (using np.linalg.solve): [ 2.  1. -2.]
Solution (via inverse): [ 2.  1. -2.]
Solutions equal? True
```